

FintelPulse — Full Build Plan

An AI equity-research web app with a light data-engineering layer. Free-forever stack. Built one working layer at a time so you always have something you can demo.

The honest framing (keep this everywhere): this is an *educational research-synthesis tool*, not a profit oracle. Next-day stock direction is close to a coin flip. The strong part of the project is that it *keeps an honest scorecard of itself* — that self-evaluation loop is your headline.

The whole thing in one paragraph

A user asks about a stock. Code we call the *orchestrator* works out the ticker, checks a *cache* for a recent answer, and serves it instantly if it's there. If not, five AI *specialists* (news, fundamentals, technical, ownership, risk) run in parallel using free live data, a *Portfolio Manager* merges them into one structured verdict, and we save that verdict to the cache and log it as a *prediction*. Every night a scheduled job checks what those stocks actually did and updates an *accuracy* scorecard. Every morning a second scheduled job builds the homepage's "biggest movers" and "most viewed" lists. That's it.

The stack (all free — verify each tier when you sign up, they drift)

Piece	Tool	What it's for	Account needed?
Language	Python 3.12	everything	no

Piece	Tool	What it's for	Account needed?
AI brain	Google Gemini (free tier)	the agents' reasoning	yes — API key
Market data	yfinance (Python library)	prices, fundamentals, ownership, outcome checks	no
News	Google News RSS	headlines for the news agent	no
Data store	Postgres on Neon (free tier)	the 5 tables	yes
Scheduler	GitHub Actions	nightly + daily jobs	yes (GitHub)
Frontend + host	Streamlit Community Cloud	the website	yes (via GitHub)

Why these (say-it-out-loud version): "I needed a hosted database because my web app and my scheduled jobs both share the same data, so a local file wouldn't work — that's Neon. Gemini's free tier is the most generous for an agent app. yfinance and Google News RSS are free with no keys. GitHub Actions is a free scheduler I already get with the repo. Streamlit lets me ship a Python web app with no front-end work."

Honest caveat to keep ready: the data is good enough for an educational project, not institutional-grade. yfinance is an unofficial feed (occasionally flaky), free news is thinner than paid wires, and ownership data is sparse for small or non-US stocks. Saying this *out loud* is the mature answer, not a weakness.

Phase 0 — Setup (accounts, keys, repo, folders)

Goal: every account created, every key stored safely, an empty repo and folder structure ready. No app logic yet.

0.1 Create the accounts and grab the keys

1. **Gemini key** — go to Google AI Studio (`aistudio.google.com`), sign in with a Google account, click **Get API Key**. No credit card. Free tier is roughly **1,500 requests/day, 15 requests/minute**. Use model `gemini-2.5-flash` (cheap, fast) — confirm the exact model name shown in AI Studio, since Google rotates these. Copy the key somewhere safe for a moment. *(Note: on the free tier Google may use your prompts to improve their models. That's fine here — you're only sending public market data, never anything private.)*
2. **Neon (Postgres)** — go to `neon.com`, sign up (no credit card), create a project. Free tier gives **0.5 GB storage and 100 compute-hours/month**, and it **scales to zero** when idle. Copy the **connection string** (looks like `postgresql://user:pass@host/dbname`). *(Scale-to-zero means the very first query after a quiet spell takes an extra second to wake — harmless for a demo.)*
3. **GitHub** — you likely have this. Create a new **public** repo named `fintelpulse`. Keep it public: public repos get effectively unlimited free GitHub Actions minutes, and it doubles as your portfolio.
4. **Streamlit Community Cloud** — go to `share.streamlit.io` and sign in **with your GitHub account**. Nothing to deploy yet; you're just creating the account for later.

0.2 The golden rule about keys

Never put a key in your code or commit it to GitHub. Keys live in three places, one per environment:

- **Locally:** a file called `.env` (which you never commit).

- **On Streamlit:** the app's *Secrets* box (paste a `secrets.toml`).
- **In GitHub Actions:** repo *Settings* → *Secrets and variables* → *Actions*.

Your code reads them from the environment, so the same code works in all three.

0.3 Folder structure

```
fintelpulse/
├─ app.py                # the Streamlit web app (live request path)
├─ core/
│   ├─ __init__.py
│   ├─ agent.py          # the reusable Agent class
│   ├─ team.py           # the 5 specialists + Portfolio Manager
│   ├─ tools.py          # yfinance + news fetchers (raw data only)
│   ├─ orchestrator.py   # resolve ticker, cache check, dispatch, save
│   └─ db.py             # Postgres connection + read/write helpers
├─ jobs/
│   ├─ nightly_accuracy.py # scheduled: fill outcomes, compute accuracy
│   └─ daily_discovery.py  # scheduled: build movers + most-viewed lists
├─ sql/
│   └─ schema.sql         # CREATE TABLE for all 5 tables
├─ .github/workflows/
│   ├─ nightly.yml        # cron for nightly_accuracy.py
│   └─ daily.yml          # cron for daily_discovery.py
├─ .env                  # local secrets – NEVER commit
├─ .gitignore            # must list .env
├─ requirements.txt
└─ README.md
```

0.4 Local environment

```
bash
```

```
git clone https://github.com/<you>/fintelpulse.git
cd fintelpulse
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
```

requirements.txt:

```
streamlit
google-genai
yfinance
psycpg2-binary
feedparser
python-dotenv
pandas
```

bash

```
pip install -r requirements.txt
```

.gitignore (at minimum):

```
.venv/
.env
__pycache__/
```

.env (local only):

```
GEMINI_API_KEY=your_key_here
DATABASE_URL=your_neon_connection_string_here
```

What you can show at the end of Phase 0: a clean repo, installed dependencies, and keys that load. Tiny check:

```
python

import os; from dotenv import load_dotenv
load_dotenv(); print(bool(os.getenv("GEMINI_API_KEY")), bool(os.getenv("DATABAS
```

Phase 1 — One agent, end to end (the vertical slice)

Goal: one specialist agent that takes a ticker, pulls real data, asks Gemini, and returns a verdict — shown on a minimal Streamlit page. No cache, no database yet.

Why this is first: it proves the hardest, riskiest part (AI + live data + a working web page) on day one. After this you have something real to click. Everything later is *adding layers around* this slice.

1.1 A raw-data tool (no opinions, just facts)

`core/tools.py`:

```
python
```

```
import yfinance as yf

def get_fundamentals(ticker: str) -> dict:
    """Return raw numbers only. No analysis here."""
    info = yf.Ticker(ticker).info
    return {
        "name": info.get("shortName"),
        "pe_ratio": info.get("trailingPE"),
        "profit_margin": info.get("profitMargins"),
        "revenue_growth": info.get("revenueGrowth"),
        "debt_to_equity": info.get("debtToEquity"),
    }
```

1.2 The reusable Agent class

This is the pattern from the workshop, rewritten for Gemini. One class, reused for every specialist — only the name, role, and system prompt change.

core/agent.py:

```
python
```

```
import os, json
from google import genai

client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])

class Agent:
    def __init__(self, name, system_prompt, model="gemini-2.5-flash"):
        self.name = name
        self.system_prompt = system_prompt
        self.model = model

    def run(self, user_content: str) -> str:
        resp = client.models.generate_content(
            model=self.model,
            contents=f"{self.system_prompt}\n\n{user_content}",
        )
        return resp.text
```

1.3 Wire one specialist + a minimal page

app.py:

```
python
```



```
import streamlit as st
from core.tools import get_fundamentals
from core.agent import Agent

st.title("FintelPulse")
st.caption("Educational research only – not financial advice.")

ticker = st.text_input("Stock ticker (e.g. AAPL, RELIANCE.NS)")
if st.button("Analyze") and ticker:
    facts = get_fundamentals(ticker)
    analyst = Agent(
        name="Fundamentals",
        system_prompt=(
            "You are a fundamentals analyst. Given raw financial numbers, "
            "give a short, balanced read: stance (bullish/neutral/bearish), "
            "a confidence (low/medium/high), and 2-3 plain reasons. "
            "End with: 'Educational only, not financial advice.'"
        ),
    )
    st.write(analyst.run(f"Ticker {ticker}. Numbers: {facts}"))
```

bash

```
streamlit run app.py
```

What you can show: a working web page where you type a ticker and get a real AI fundamentals read on live data. **This is already demoable.**

Say-it-in-an-interview: "I built a thin vertical slice first — one agent, real data, a live page — so I had a working product before adding any infrastructure."

Phase 2 — The data store: cache + prediction log + view counter

Goal: stand up Postgres, add the cache so repeated questions are instant and free, log every fresh analysis as a prediction, and count views. This is your first real data-engineering layer.

Why now: before adding more agents, give the system a memory. The cache makes it fast and cheap; the prediction log is what the accuracy scorecard will later score.

2.1 Create the tables

sql/schema.sql:

```
sql
```

```

CREATE TABLE IF NOT EXISTS verdict_cache (
    ticker      TEXT PRIMARY KEY,
    verdict     JSONB NOT NULL,
    made_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS predictions (
    id          SERIAL PRIMARY KEY,
    ticker      TEXT NOT NULL,
    predicted_on DATE NOT NULL,
    prediction   TEXT NOT NULL,          -- bullish / neutral / bearish
    for_date     DATE NOT NULL,          -- the day we're predicting
    actual_move  NUMERIC,                 -- filled by the nightly job
    correct      BOOLEAN                  -- filled by the nightly job
);

CREATE TABLE IF NOT EXISTS accuracy (
    scope        TEXT NOT NULL,          -- 'overall' or an agent name
    period        TEXT NOT NULL,         -- e.g. 'last_30_days'
    hit_rate      NUMERIC NOT NULL,
    total         INT NOT NULL,
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (scope, period)
);

CREATE TABLE IF NOT EXISTS discovery_list (
    kind         TEXT NOT NULL,          -- 'movers' or 'most_viewed'
    payload       JSONB NOT NULL,        -- the ranked list
    built_on      DATE NOT NULL,
    PRIMARY KEY (kind, built_on)
);

```

```
CREATE TABLE IF NOT EXISTS ticker_views (  
    ticker      TEXT PRIMARY KEY,  
    views       INT NOT NULL DEFAULT 0,  
    last_asked  TIMESTAMPTZ NOT NULL DEFAULT now()  
);
```

Run it once against Neon (via the Neon web SQL editor, or psql with your connection string).

2.2 DB helpers

core/db.py:

```
python
```

```
import os, json, psycopg2

def conn():
    return psycopg2.connect(os.environ["DATABASE_URL"])

def get_cached(ticker, ttl_hours=4):
    with conn() as c, c.cursor() as cur:
        cur.execute("""
            SELECT verdict FROM verdict_cache
            WHERE ticker = %s AND made_at > now() - interval '%s hours'
            """, (ticker, ttl_hours))
        row = cur.fetchone()
        return row[0] if row else None

def save_verdict_and_log(ticker, verdict: dict, for_date):
    with conn() as c, c.cursor() as cur:
        cur.execute("""
            INSERT INTO verdict_cache (ticker, verdict, made_at)
            VALUES (%s, %s, now())
            ON CONFLICT (ticker) DO UPDATE SET verdict = EXCLUDED.verdict, made_at = EXCLUDED.made_at
            """, (ticker, json.dumps(verdict)))
        cur.execute("""
            INSERT INTO predictions (ticker, predicted_on, prediction, for_date)
            VALUES (%s, current_date, %s, %s)
            """, (ticker, verdict["stance"], for_date))

def bump_view(ticker):
    with conn() as c, c.cursor() as cur:
        cur.execute("""
            INSERT INTO ticker_views (ticker, views, last_asked)
            VALUES (%s, 1, now())
            ON CONFLICT (ticker) DO UPDATE
        """)
```

```
        SET views = ticker_views.views + 1, last_asked = now()
    """ , (ticker,))
```

2.3 The flow in the orchestrator

`core/orchestrator.py` (the shape):

```
python

from core import db

def answer(ticker, run_agents):  # run_agents() returns a verdict dict
    db.bump_view(ticker)          # event: +1 view, every ask
    cached = db.get_cached(ticker)
    if cached:
        return cached           # HIT: instant, ~$0
    verdict = run_agents(ticker)  # MISS: do the work
    db.save_verdict_and_log(ticker, verdict, for_date=tomorrow())
    return verdict
```

What you can show: ask the same ticker twice — second time is instant (a cache hit). Open Neon and see rows appearing in `predictions` and `ticker_views`.

Say-it-in-an-interview: "On a cache miss I write the verdict and log a prediction in the same step — the cache makes repeats instant, and the log is the raw material for the accuracy scorecard."

Phase 3 — The full agent team (5 specialists + manager, in parallel)

Goal: replace the single agent with the real team — five independent specialists running in

parallel, feeding one Portfolio Manager that returns a structured verdict.

Why parallel: the specialists never read each other's output, so running them at once gives the identical result, faster. Only the manager waits, because it combines them.

3.1 News tool (raw headlines only)

`core/tools.py` (add):

```
python

import feedparser, urllib.parse

def get_headlines(query: str, limit=8):
    url = "https://news.google.com/rss/search?q=" + urllib.parse.quote(query)
    feed = feedparser.parse(url)
    return [e.title for e in feed.entries[:limit]] # the agent judges sentiment
```

3.2 The team, run in parallel

`core/team.py` (the shape):

```
python
```

```

import json
from concurrent.futures import ThreadPoolExecutor
from core.agent import Agent
from core import tools

SPECIALISTS = {
    "news":          ("News analyst...",          lambda t: tools.get_headlines(t))
    "fundamentals":  ("Fundamentals analyst...", lambda t: tools.get_fundamental
    "technical":     ("Technical analyst...",      lambda t: tools.get_price_histor
    "ownership":     ("Ownership analyst...",      lambda t: tools.get_ownership(t)
    "risk":          ("Risk analyst...",           lambda t: tools.get_everything(t)
}

def _run_one(name, ticker):
    prompt, fetch = SPECIALISTS[name]
    agent = Agent(name=name, system_prompt=prompt)
    return name, agent.run(f"Ticker {ticker}. Data: {fetch(ticker)}")

def run_team(ticker):
    with ThreadPoolExecutor(max_workers=5) as pool:          # 5 at once
        reports = dict(pool.map(lambda n: _run_one(n, ticker), SPECIALISTS))
    pm = Agent(
        name="PortfolioManager",
        system_prompt=(
            "Combine the five analyst reports into ONE verdict. "
            "Return ONLY JSON: {\"stance\": \"...\", \"confidence\": \"...\", \""
            "\"drivers\": [\"...\"]}. No prose, no code fences."
        ),
    )

```



```
raw = pm.run(json.dumps(reports))
return json.loads(raw.replace("```json", "").replace("```", "").strip())
```

Note on the free tier: 5 specialists + 1 manager = 6 calls per analysis, under the 15-requests/minute limit for one user. The cache absorbs bursts from repeated tickers. If you ever hit the limit, lower `max_workers` to run a few at a time.

What you can show: a real multi-agent verdict with named drivers, returned as clean JSON and saved to the cache.

Say-it-in-an-interview: "Five independent specialists run in parallel; only the synthesizer waits because it depends on all of them. Same answer as sequential, about five times faster."

Phase 4 — The nightly job (the accuracy pipeline) ← your headline DE feature

Goal: a scheduled job that, each night, reads yesterday's predictions, checks what those stocks *actually* did, writes the outcomes back, and recomputes the accuracy scorecard.

Why this is the headline: this is the honest self-evaluation loop. It's also a textbook data pipeline — scheduled ingestion of outcome data plus metric computation.

4.1 The job

`jobs/nightly_accuracy.py` (the shape):

```
python
```

```
import yfinance as yf
from core.db import conn

def resolve_outcomes():
    with conn() as c, c.cursor() as cur:
        cur.execute("SELECT id, ticker, prediction, for_date FROM predictions W
        for pid, ticker, prediction, for_date in cur.fetchall():
            move = pct_change_on(ticker, for_date)          # via yfinance
            correct = (move > 0 and prediction == "bullish") or \
                      (move < 0 and prediction == "bearish")
            cur.execute("UPDATE predictions SET actual_move=%s, correct=%s WHERE
                        (move, correct, pid))

def recompute_accuracy():
    with conn() as c, c.cursor() as cur:
        cur.execute("""
            INSERT INTO accuracy (scope, period, hit_rate, total, updated_at)
            SELECT 'overall', 'last_30_days',
                   AVG(CASE WHEN correct THEN 1 ELSE 0 END), COUNT(*), now()
            FROM predictions
            WHERE correct IS NOT NULL AND for_date > current_date - 30
            ON CONFLICT (scope, period) DO UPDATE
            SET hit_rate = EXCLUDED.hit_rate, total = EXCLUDED.total, updated_a
        """)

if __name__ == "__main__":
    resolve_outcomes()
    recompute_accuracy()
```

`.github/workflows/nightly.yml`:

```
yaml
name: nightly-accuracy
on:
  schedule:
    - cron: "30 18 * * *"      # daily; pick a time after market close (UTC)
  workflow_dispatch:          # lets you run it by hand to test
jobs:
  run:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: python -m jobs.nightly_accuracy
    env:
      GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
      DATABASE_URL: ${ secrets.DATABASE_URL }
```

Add `GEMINI_API_KEY` and `DATABASE_URL` under repo **Settings** → **Secrets and variables** → **Actions**. Test with the **Run workflow** button (that's `workflow_dispatch`) before trusting the schedule.

What you can show: make a prediction today, run the job by hand, watch the `predictions` row fill in `actual_move` + `correct`, and the `accuracy` row appear.

Say-it-in-an-interview: "A scheduled GitHub Actions pipeline ingests each prediction's real next-day outcome and computes a rolling 30-day accuracy. The product reports its own

honest hit-rate instead of pretending to be right."

Phase 5 — Discovery panel + daily job

Goal: fix the empty homepage. A morning job builds two lists — biggest movers and most-viewed — and the homepage reads them. **Zero AI** in this whole phase.

Why it's cheap: these lists are *facts you read off* (a price change, a view count), not opinions. No agent runs. Agents only run when a user actually clicks a stock.

5.1 The daily job

`jobs/daily_discovery.py` (the shape):

```
python
```

```

import json, yfinance as yf
from core.db import conn

UNIVERSE = ["RELIANCE.NS", "INFY.NS", "TCS.NS", "HDFCBANK.NS", "ITC.NS"] # you

def build_movers():
    data = yf.download(UNIVERSE, period="2d")["Close"] # ONE batched call
    pct = ((data.iloc[-1] / data.iloc[-2]) - 1) * 100
    ranked = sorted(pct.items(), key=lambda kv: kv[1], reverse=True)
    return [{"ticker": t, "pct": round(p, 2)} for t, p in ranked]

def build_most_viewed():
    with conn() as c, c.cursor() as cur:
        cur.execute("SELECT ticker, views FROM ticker_views ORDER BY views DESC")
        return [{"ticker": t, "views": v} for t, v in cur.fetchall()]

def save(kind, payload):
    with conn() as c, c.cursor() as cur:
        cur.execute("""
            INSERT INTO discovery_list (kind, payload, built_on)
            VALUES (%s, %s, current_date)
            ON CONFLICT (kind, built_on) DO UPDATE SET payload = EXCLUDED.payload
            """, (kind, json.dumps(payload)))

if __name__ == "__main__":
    save("movers", build_movers())
    save("most_viewed", build_most_viewed())

```

`.github/workflows/daily.yml` is the same as `nightly.yml` but with a morning cron (e.g. `"0 3 * * *`) and `(python -m jobs.daily_discovery.`

5.2 The homepage reads the lists

In `app.py`, before the search box, read the latest `discovery_list` rows and the `accuracy` row, and render the two panels + a "52%" badge. Each list row links to the same `answer()` flow — and popular tickers are usually cache hits, so clicks are mostly instant.

Frame it safely: "biggest movers today" and "most viewed here" are factual and descriptive. Never label anything "stocks to buy" — that turns a project into financial advice.

What you can show: a homepage that greets visitors with what's moving and what's popular, plus your honest accuracy badge, all built by a free daily job with no AI.

Say-it-in-an-interview: "The discovery lists are built from cheap market data and my own usage counter — zero AI. Agents only run on a real user question, and the cache absorbs the popular repeats."

Phase 6 — Deploy + polish + the narrative

Goal: put it online and make it interview-ready.

1. **Deploy:** at `share.streamlit.io`, click **Create app**, point it at your repo and `app.py`. Paste your keys into the **Secrets** box as `secrets.toml`:

toml

```
GEMINI_API_KEY = "..."  
DATABASE_URL = "..."
```

(Read them in code with `st.secrets` or keep using `os.environ` — Streamlit exposes both.) 2. **Disclaimers:** "educational only — not financial advice" on the homepage, on every verdict, and in the README. 3. **README:** the one-paragraph summary, the architecture diagram, the honest data caveats, and the honest accuracy result. *Reporting ~50% honestly*

is a stronger story than claiming you beat the market. 4. **The story you tell:** "I built an AI research desk with a light data-engineering layer. The interesting engineering isn't the prediction — it's the honest evaluation loop: a scheduled pipeline that scores every prediction against reality and reports a real hit-rate. I deliberately skipped a data lake and dbt because they'd have been over-engineering at this scope."

Build-order cheat sheet

Phase	You add	You can demo
0	accounts, keys, repo, folders	keys load
1	one agent + minimal page	type a ticker → AI read on live data
2	Postgres: cache, predictions, views	repeats are instant; rows appear
3	full team, parallel + manager	structured multi-agent verdict
4	nightly accuracy pipeline	outcomes fill in; scorecard appears
5	discovery panel + daily job	homepage with movers + most-viewed
6	deploy + polish	a public URL + an interview story

A few things to verify as you go

- Confirm the exact Gemini model name in AI Studio (Google rotates them).
- Re-check free-tier limits at signup (Gemini RPM/RPD, Neon storage/compute) — they drift.

- yfinance is unofficial; wrap data calls in try/except so one flaky fetch doesn't crash the app.
- Keep the repo public so GitHub Actions stays free.